



EMBEDDED SECURITY

Cryptography Lib Lab — Path 2: Algorithm Comparison

CryptoLab

AES-128-CBC vs DES-56-CBC

MennatAllah Essam Mohamed

Section 2 | 14 May 2026

PyCryptodome · Flask · PKCS#7 · CBC Mode

Table of Contents

Table of Contents	2
1. Scenario & Objectives.....	3
2. System Overview	3
3. Algorithms & Implementation.....	3
4. Keys, IV & Plaintext Format	3
Key	3
IV.....	3
Plaintext.....	3
5. Encrypted Output	4
6. Decrypted Output.....	4
7. Verification of Correct Decryption	4
8. API Summary	4
9. Results & Observations	4
Timing.....	4
Size	4
Security.....	4
10. Security Limitations	5
11. Figures — Screenshots	5
Figure 1 — Application Overview	5
Figure 2 — Message Editor.....	5
Figure 3 — Ciphertext Output	5
Figure 4 — Metrics & Security Comparison	6
Figure 5 — Decrypt & Verified Output	7
12. How to Run.....	7

1. Scenario & Objectives

CryptoLab is a Flask web app where users compose chat messages, then encrypt them with AES-128-CBC and DES-56-CBC simultaneously to compare both ciphers on identical plaintext.

- Encrypt the same JSON payload with AES-128 and DES using CBC mode + PKCS#7 padding.
- Display ciphertext (hex, Base64, binary), keys, and IVs side-by-side.
- Measure and compare encryption/decryption times and ciphertext sizes.
- Verify bit-exact round-trip recovery for both algorithms.
- Persist output artifacts to output/ on disk.

2. System Overview

Layer	Technology	Role
Frontend	HTML / CSS / JS	Message editor, result cards, charts, copy/download.
Backend	Flask + PyCryptodome	/api/encrypt, /api/decrypt — runs AES & DES, writes output/.
Sample data	data/original_messages.json	Default 5-message chat payload served on load.
Artifacts	output/	Ciphertext (.bin/.hex/.b64), plaintext.json, decrypted JSON.

Flow: User edits messages → POST /api/encrypt → server encrypts with both ciphers, verifies round-trip, writes output/, returns keys/IVs/ciphertext/timings → User optionally calls POST /api/decrypt to recover messages.

3. Algorithms & Implementation

Algorithm	Key / Block	Mode	Status
AES-128	128-bit key / 16 B block	CBC + PKCS#7	NIST standard — SECURE
DES-56	56-bit key / 8 B block	CBC + PKCS#7	Deprecated 2005 — LEGACY

- Library: PyCryptodome (Crypto.Cipher.AES / DES, Crypto.Util.Padding).
- Both use MODE_CBC; IV is randomly generated per run and returned alongside ciphertext.
- Production note: prefer AES-GCM (AEAD) — CBC alone provides no integrity protection.

4. Keys, IV & Plaintext Format

Key

- AES: 16 random bytes (2^{128} combinations). DES: 8 random bytes (2^{56} effective).
- Fresh key generated per request via `get_random_bytes` — never reused.

IV

- Created automatically by the library on cipher instantiation, read from `cipher.iv`.
- Returned as `iv_hex` in the API; must be supplied unchanged for decryption.

Plaintext

```
json.dumps(messages, ensure_ascii=False).encode("utf-8")
```

The entire chat array is serialized as one UTF-8 JSON document before encryption.

5. Encrypted Output

Metric	AES-128-CBC	DES-56-CBC
Plaintext size	682 B	682 B
Ciphertext size	720 B	712 B
Padding overhead	+38 B (pad to 16 B boundary)	+30 B (pad to 8 B boundary)
Key size	128 bits	64 bits (56 effective)
Block size	16 B	8 B

- Hex encoding $\approx 2\times$ raw byte length; Base64 $\approx 4/3\times$ — both available for copy/download.
- Server writes .bin, .hex.txt, .b64.txt to output/ after every successful encrypt.

6. Decrypted Output

- UI: recovered messages shown as cards + pretty-printed JSON panel with Copy / Download.
- Disk: output/decrypted_aes.json and output/decrypted_des.json (overwritten per run).
- API returns: messages, dec_time_ms, algorithm, output_saved.

7. Verification of Correct Decryption

The server automatically decrypts immediately after encrypting and performs a byte-exact comparison:

```
verify_files(plaintext, decrypted_data) # returns True if original_data == decrypted_data
```

- Result returned as verified: true / false per algorithm in the API response.
- Both AES and DES consistently returned verified: true across all test runs.
- UI shows a "Verified" badge on the Decrypt panel confirming round-trip integrity.

8. API Summary

Method	Endpoint	Key Response Fields
GET	/api/messages	messages (from original_messages.json)
POST	/api/encrypt	results.{AES,DES}: ciphertext_hex/b64, key_hex, iv_hex, timings, sizes, verified
POST	/api/decrypt	messages, dec_time_ms, algorithm, output_saved

9. Results & Observations

Timing

Both ciphers completed in < 1 ms for the 682-byte payload — no measurable difference at this size. At scale, AES benefits from hardware AES-NI acceleration; DES runs in pure software.

Size

AES produced 720 B (38 B padding overhead) vs DES at 712 B (30 B overhead). The slightly larger AES output reflects its 16 B block size vs DES's 8 B block.

Security

AES-128-CBC (A+)	DES-56-CBC (F)
2^{128} key combinations	Crackable in < 24 hours

NIST-approved, used in TLS/SSH/VPN	Deprecated by NIST since 2005
Resistant to all known attacks	Vulnerable to brute-force & differential cryptanalysis

10. Security Limitations

- Keys & IVs returned in plaintext API response — acceptable for learning, never for production.
- DES 56-bit key space is brute-forceable; do not use for any new confidential data.
- CBC mode lacks authentication — ciphertext is malleable without HMAC or AEAD.
- Disk artifacts (output/) may contain key material — secure the server, do not commit to public repos.

11. Figures — Screenshots

Figure 1 — Application Overview

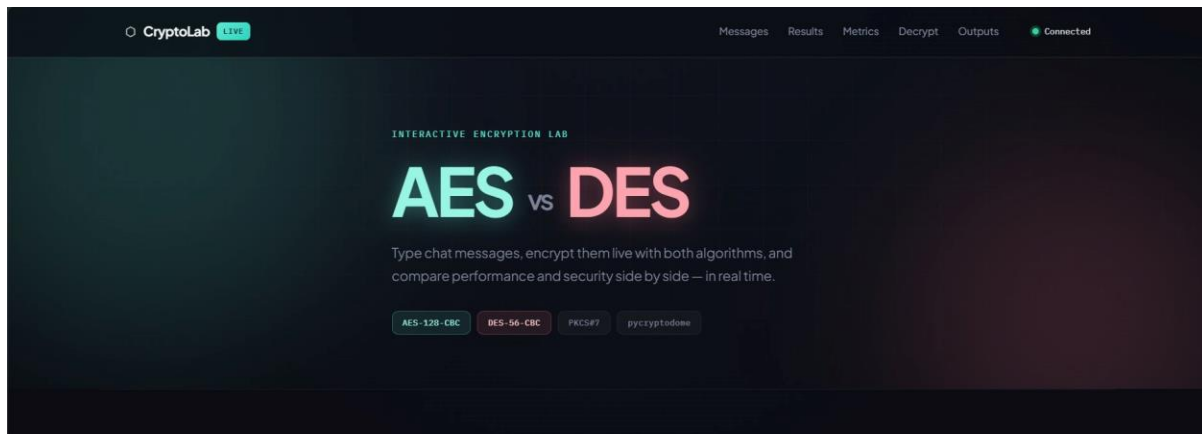


Fig. 1: CryptoLab home — AES vs DES branding, navigation, algorithm tags.

Figure 2 — Message Editor

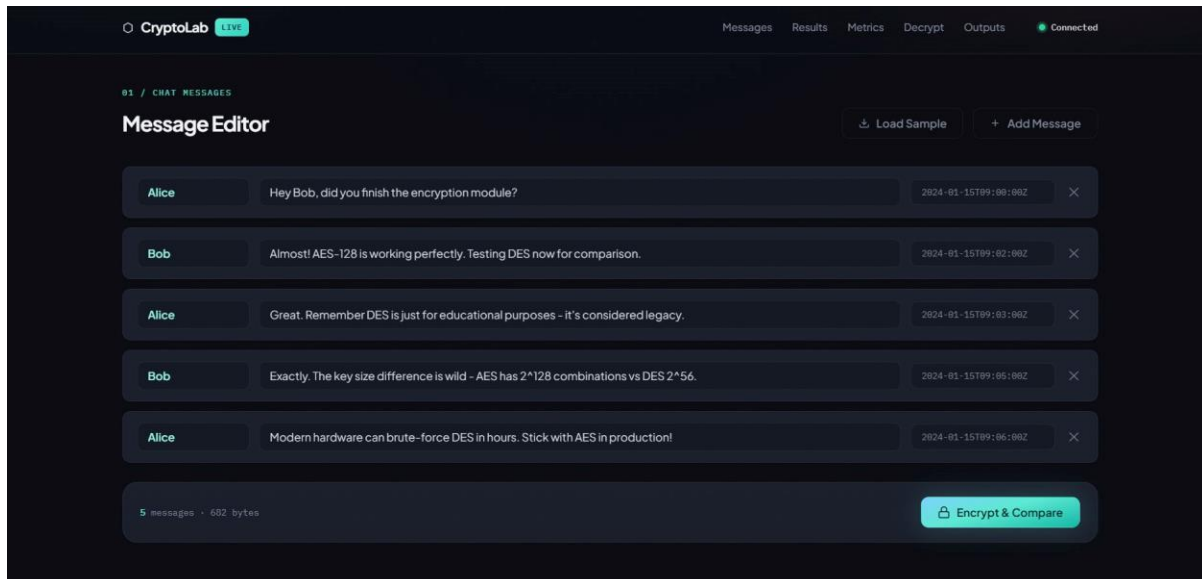


Fig. 2: Message editor — 5 sample messages, 682 bytes, Encrypt & Compare button.

Figure 3 — Ciphertext Output

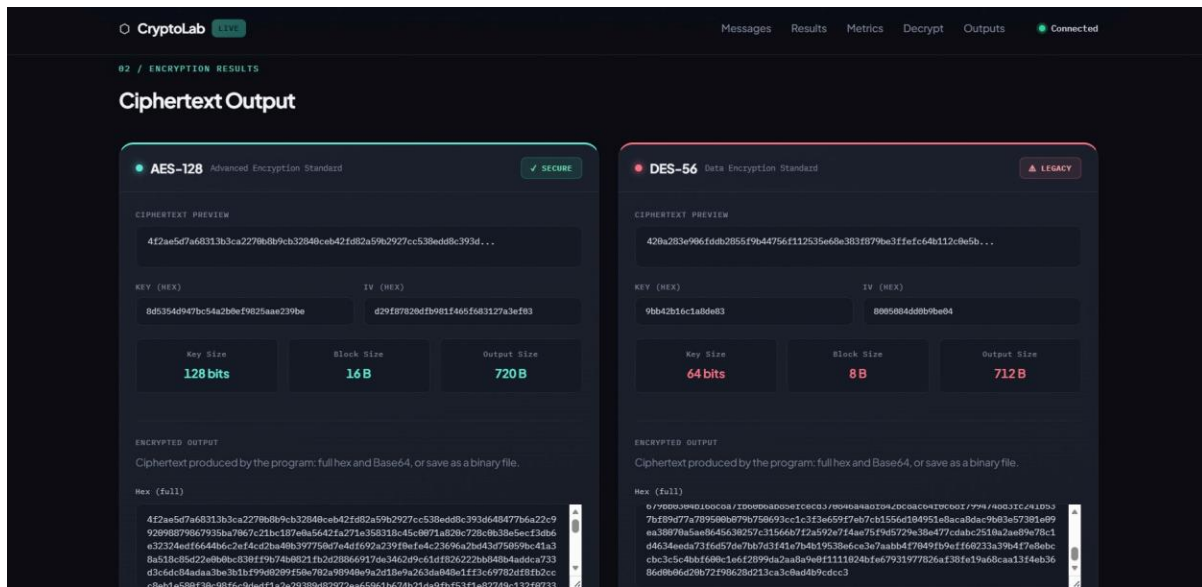


Fig. 3a: Side-by-side AES (SECURE) and DES (LEGACY) result cards — preview, key/IV, sizes.

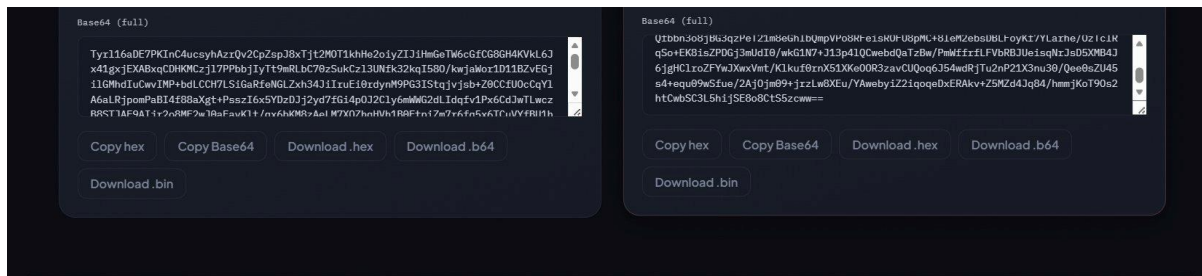


Fig. 3b: Full Base64 output panels with Copy hex/b64 and Download .bin buttons.

Figure 4 — Metrics & Security Comparison

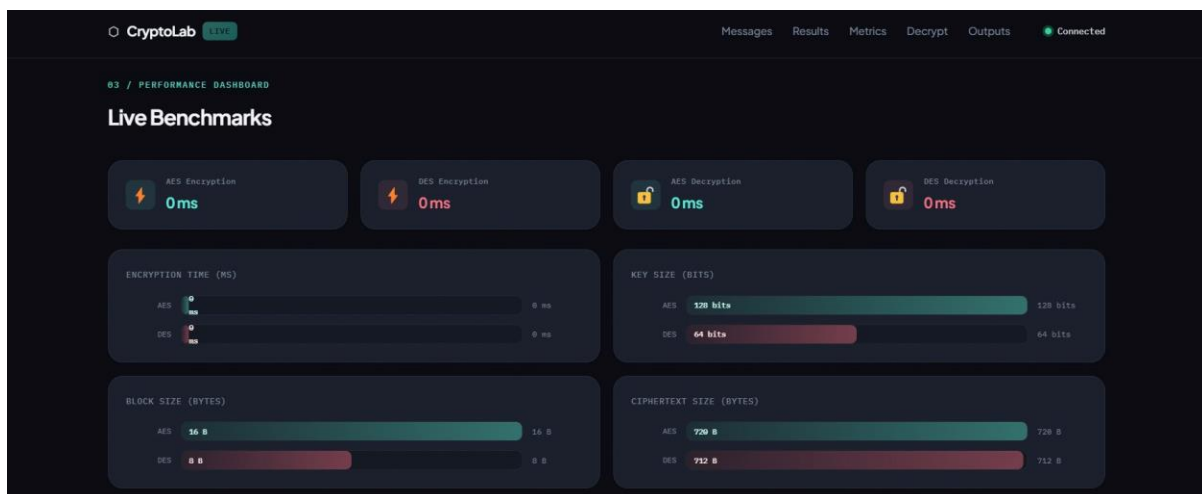


Fig. 4a: Live Benchmarks — timing cards (< 1 ms), key/block/ciphertext size bar charts.

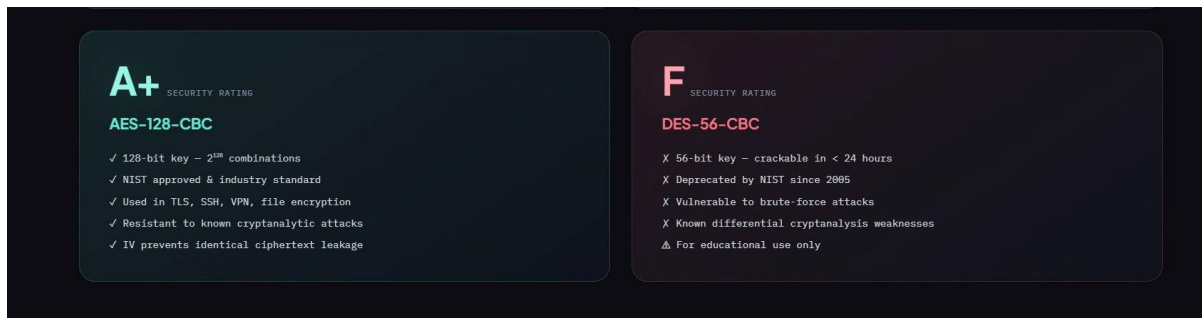


Fig. 4b: Security ratings — AES A+ vs DES F with detailed comparison points.

Figure 5 — Decrypt & Verified Output

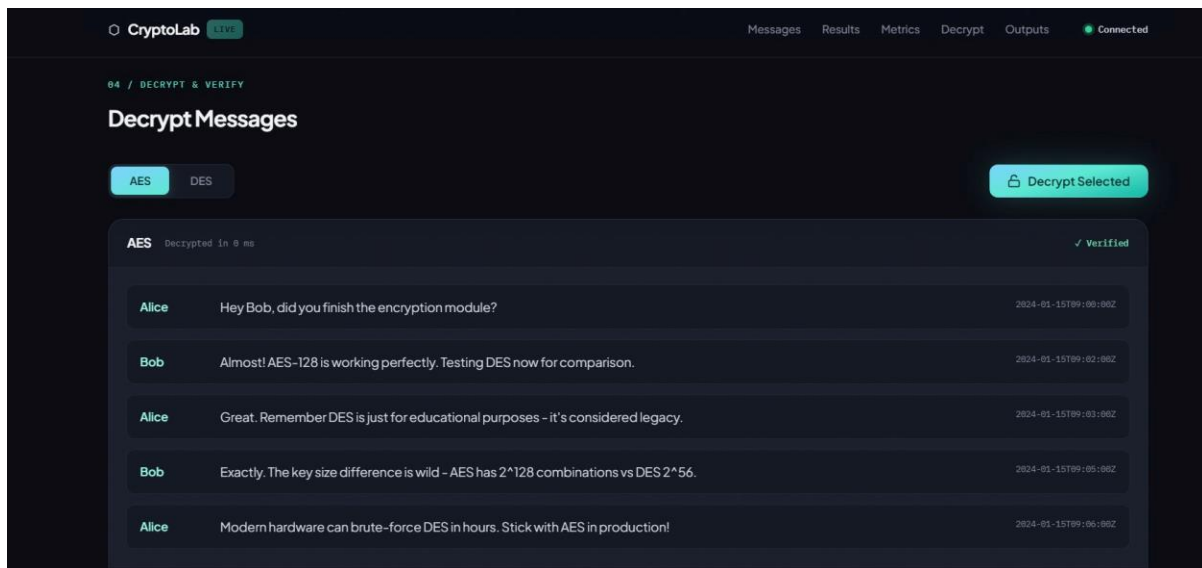


Fig. 5a: Decrypt panel — AES selected, Verified badge, all 5 messages recovered.

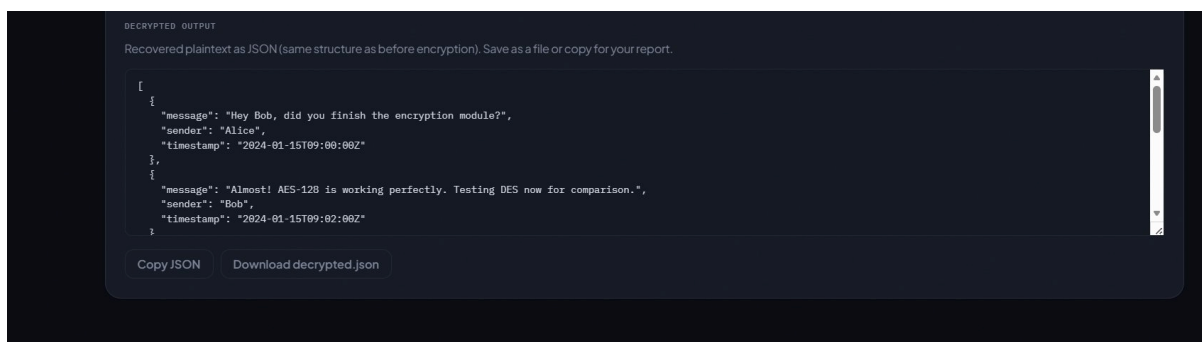


Fig. 5b: Decrypted Output — recovered JSON with Copy and Download buttons.

12. How to Run

1. Install deps: `pip install flask pycryptodome`
2. Start server: `python app.py` → `http://127.0.0.1:5000`
3. Browser: edit messages → Encrypt & Compare → inspect results → Decrypt Selected.
4. Artifacts: check output/ for .bin/.hex/.b64 ciphertext files and decrypted_*.json.

— End of Report —